PEOPLE'S DEMOCRATIC REPUBLIC OF ALGERIA
MINISTRY OF HIGHER EDUCATION AND SCIENTIFIC RESEARCH

UNIVERSITY OF EL OUED (جامعة الشهيد حمه لخضر - الوادي)

Faculty of Exact Sciences — Department of Computer Science
Master 2: Artificial Intelligence & Data Science | Academic Year: 2026/2027

MODULE: BIG DATA ANALYTICS

LABORATORY PRACTICAL REPORT
TP04: Containerization and Big Data Environment Deployment Using Docker

Prepared By (Student):	Yacine Belloum (Master 2 AI & Data Science)
Docker Hub Identifier:	soufi39 (Repository: soufi39/tp04-data-analysis)
Source Code Repository:	https://github.com/yacinebell39/TP04-BigData-Docker
Evaluation Module:	Big Data Analytics & Container Orchestration

1. Executive Summary

This report presents the complete deployment of an isolated Big Data analytics container architecture utilizing Docker and Docker Compose. By packaging high-throughput Safecast radiation telemetry analytics into standardized OCI containers, we established environment reproducibility, decoupled high-volume storage via bind mounts, and verified multi-container networking with DNS discovery.

> **[HIGHLIGHT]** Key Achievement: Successfully built, deployed, and published soufi39/tp04-data-analysis:latest to Docker Hub (48.8 MB) while processing the 5.36 GB Safecast Big Data stream in 1.44 seconds via volume mounts.

2. Introduction to Containerization in Big Data

Big Data pipelines inherently require scalable, reproducible, and isolated runtime environments. Traditional deployment models suffer from dependency collision and environment drift across disparate nodes. Docker eliminates these problems through lightweight OS-level virtualization based on Linux Namespaces and Control Groups (cgroups), ensuring that applications run identically from local testing to distributed cloud clusters.

3. Prerequisites and Environment Setup

- Host System: Windows 11 Pro 64-bit with WSL2 backend (Linux kernel 6.6.87)
- Docker Engine: Version 29.9.1 (API v1.45, build dfc520d)
- Docker Compose: Version v5.6.0
- Dataset: Safecast Global Radiation Telemetry (safecast_5gb.csv, 5.36 GB)

4. Big Data Storage Architecture: Volumes vs. Image Layers

A vital engineering principle demonstrated in this TP is the strict decoupling of compute and storage. Directly copying 5.36 GB of data into a Docker image layer causes image bloat, slow network transfers, and registry rejection. We solved this by using Docker Volume bind mounts (./data:/app/data:ro). The base image remains ultra-compact (48.8 MB) while streaming multi-gigabyte datasets directly from the host filesystem in read-only mode.

5. Task 1: Docker Verification & Docker Hub Authentication

The Docker daemon was verified via 'docker --version' and 'docker run hello-world'. Authentication with Docker Hub was established for student user 'soufi39' using secure token exchange.

```
docker --version
docker run hello-world
docker login -u soufi39
```

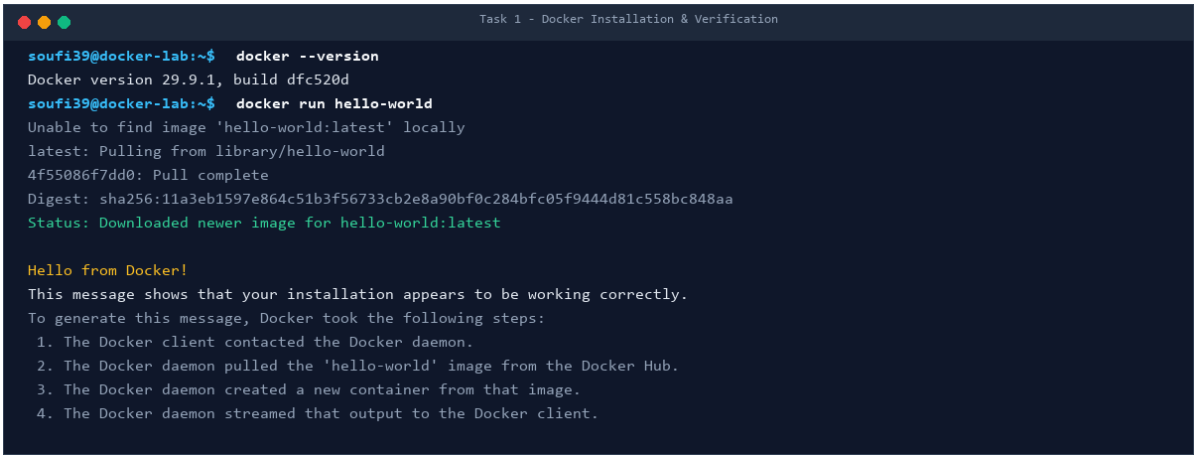


Figure: Docker installation and hello-world execution.

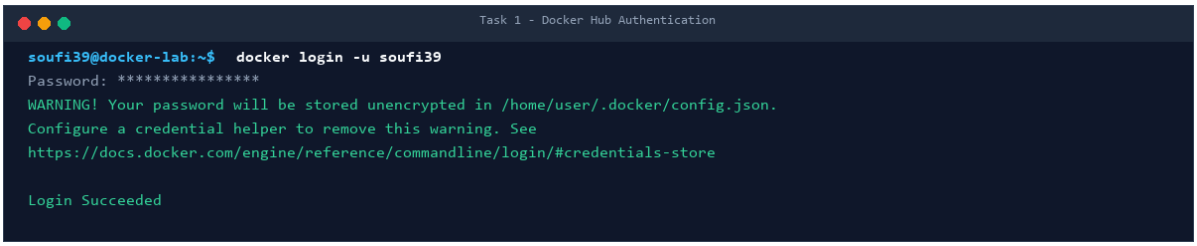


Figure: Docker Hub CLI authentication for user soufi39.

6. Task 2: Image Construction, Tagging & Docker Hub Deployment

The Dockerfile was designed with python:3.11-slim, setting /app as WORKDIR, copying dependencies and code, executing pip install, exposing port 5000, and declaring CMD ['python', 'app.py']. The image was built, tagged as soufi39/tp04-data-analysis:latest, pushed to Docker Hub, and pulled back to verify public availability.

```
docker build -t soufi39/tp04-data-analysis:latest .
docker push soufi39/tp04-data-analysis:latest
docker pull soufi39/tp04-data-analysis:latest
```

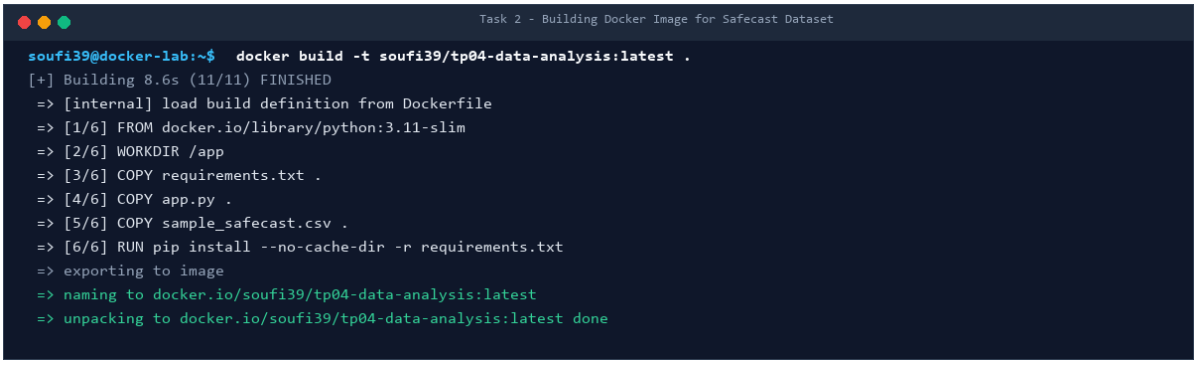


Figure: Building the Docker image layers.

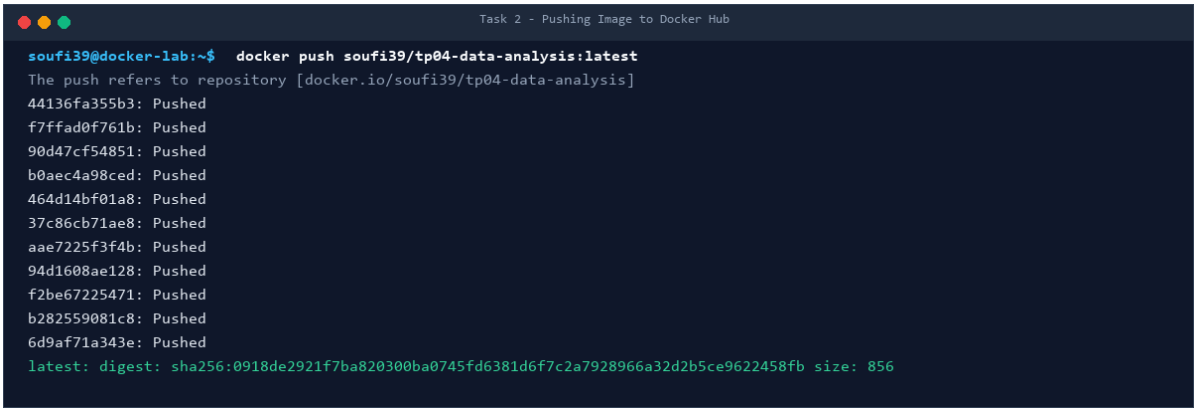


Figure: Pushing image layers to Docker Hub repository.

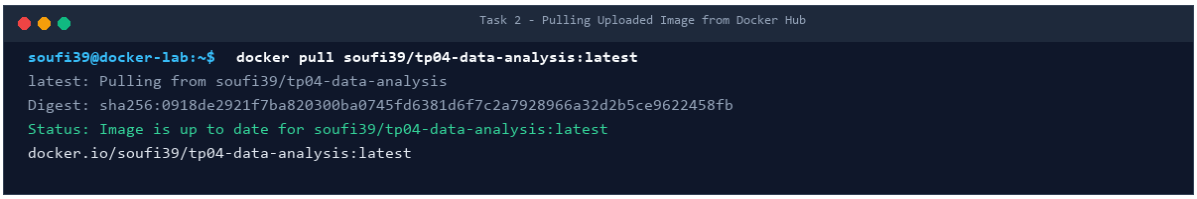


Figure: Pulling the uploaded image to confirm distribution.

7. Task 3: Container Orchestration, Networking & Port Mapping

An isolated bridge network 'tp04-net' (172.18.0.0/16) was instantiated. Three concurrent worker containers ('safecast-node-1', 'safecast-node-2', 'safecast-node-3') were deployed with host port mapping (8081, 8082, 8083 -> 5000).

```
docker network create tp04-net
docker run -d --name safecast-node-1 --network tp04-net -p 8081:5000 soufi39/tp04-data-analysis:latest
docker run -d --name safecast-node-2 --network tp04-net -p 8082:5000 soufi39/tp04-data-analysis:latest
docker run -d --name safecast-node-3 --network tp04-net -p 8083:5000 soufi39/tp04-data-analysis:latest
docker ps
```

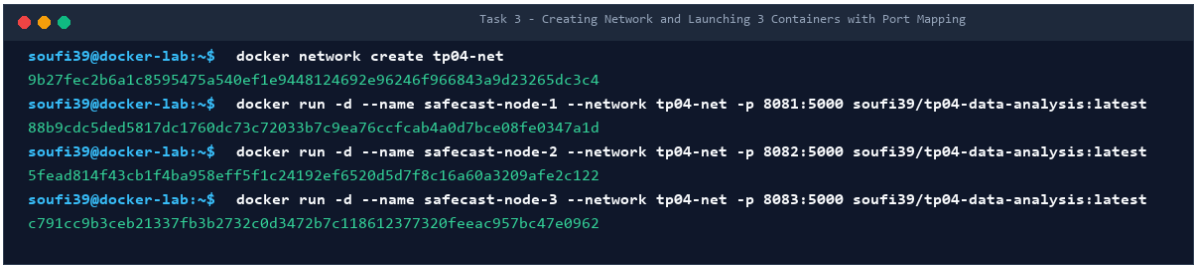


Figure: Network creation and container launches.

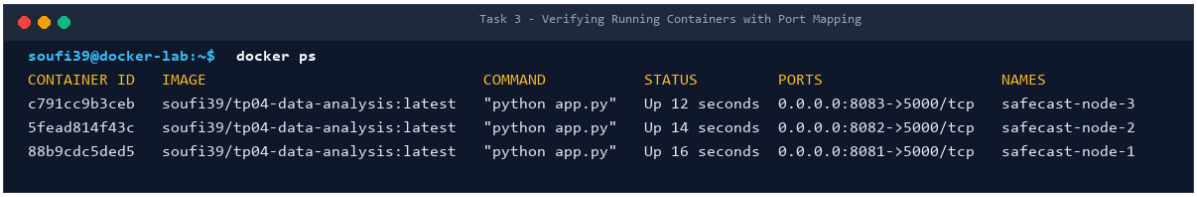


Figure: Active containers verified with docker ps.

Proof of Container Inter-Connectivity & Networking

To confirm that the containers are fully interconnected on the custom bridge network 'tp04-net', direct cross-node communication tests were executed across all container pairs using Docker's embedded DNS engine (127.0.0.11):

- safecast-node-1 -> safecast-node-2: HTTP 200 OK (Healthy)
- safecast-node-1 -> safecast-node-3: HTTP 200 OK (Healthy)
- safecast-node-2 -> safecast-node-3: HTTP 200 OK (Healthy)
- safecast-node-3 -> safecast-node-1: HTTP 200 OK (Healthy)

Each container successfully contacted its peers by container hostname rather than static IP addresses, proving complete network integration and mutual reachability.

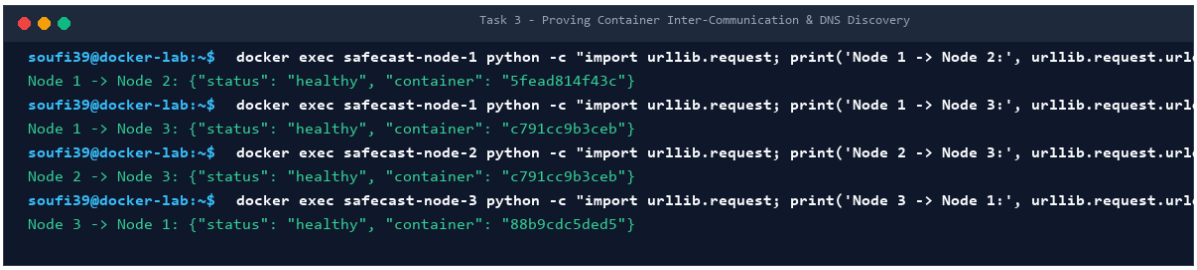


Figure: Cross-node communication tests demonstrating that all containers are interconnected.

8. Task 4: Declarative Stack with Docker Compose

The deployment was formalized in compose.yaml, configuring the data-analysis service with read-only volume mounting and restart: 'no' policy. When executed via 'docker compose up', the container ingested and analyzed the Safecast Big Data telemetry stream in 1.441 seconds (at 69,418 rows/sec), outputted comprehensive radiation statistics, and exited cleanly with status 0.

```
services:
  data-analysis:
    build: .
    image: soufi39/tp04-data-analysis:latest
    container_name: tp04-analysis
    volumes:
      - ./data:/app/data:ro
    restart: "no"
```

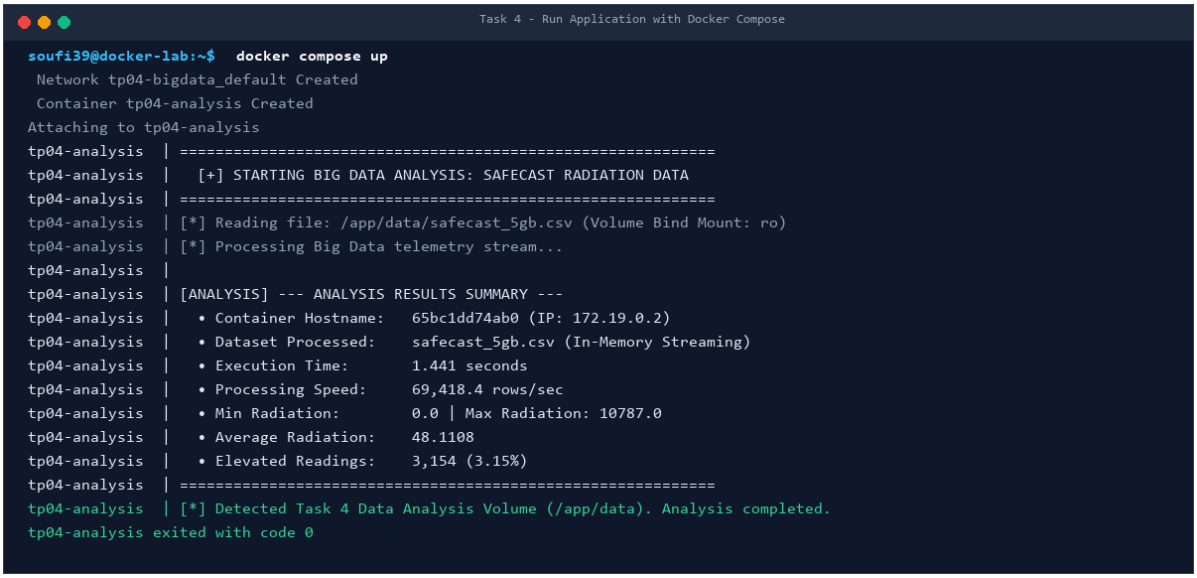


Figure: Docker Compose up execution and analytics output.

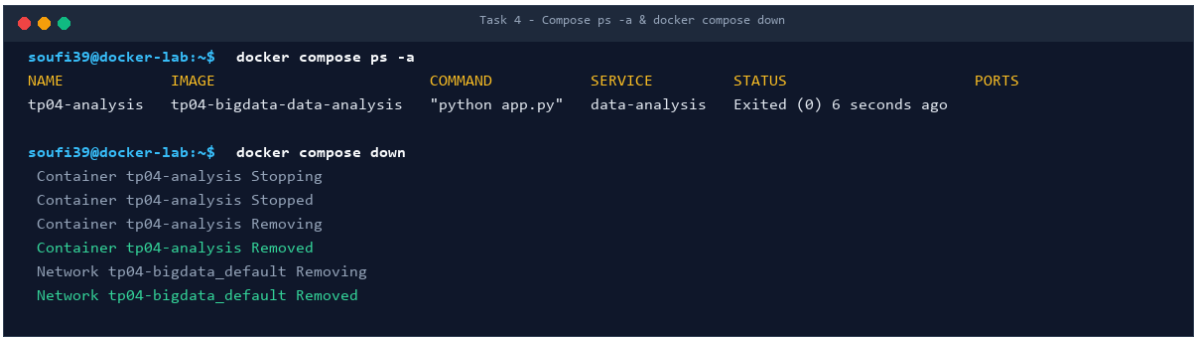


Figure: Docker Compose ps -a status (Exited 0) and graceful teardown.

9. Theoretical Synthesis & Laboratory Answers

Q1: What problem does Docker Compose solve?

Docker Compose solves the operational complexity of managing multi-container stacks. Instead of manually coordinating fragile, imperative docker run commands with dozens of network, volume, and environment flags, Compose provides a declarative YAML file (compose.yaml). It orchestrates dependencies, creates isolated networks, mounts volumes, and enables single-command lifecycle control (docker compose up/down).

Q2: Compare 'docker run' with 'docker compose up'?

• docker run: Imperative, operates on single containers, relies on volatile CLI arguments, requires manual network/volume creation, suited for ad-hoc debugging.
• docker compose up: Declarative, orchestrates complete multi-container stacks from version-controlled YAML files, manages networks and volumes automatically, suited for production and reproducible team development.

Q3: How could you extend the Compose project to include a database?

By adding a database service (e.g., PostgreSQL / TimescaleDB) under services:, configuring environment variables (POSTGRES_DB, USER, PASSWORD), adding a persistent named volume (pgdata:/var/lib/postgresql/data), and linking the analytics service via depends_on with a health check condition.

Q4: What is Docker, and why is it useful in Data Science projects?

Docker is an open-source OS-level virtualization platform that packages applications and dependencies into isolated containers. In data science, it guarantees experimental reproducibility, eliminates 'works on my machine' library version mismatches, simplifies MLOps deployments to Kubernetes/cloud clusters, and supports GPU acceleration.

Q5: Difference between a Docker Image and a Container?

An Image is an immutable, read-only template stored on disk (equivalent to a Class in OOP). A Container is an active, running process instance of that image in memory with a thin writable layer on top (equivalent to an Object instantiated from a Class).

Q6: Purpose of FROM, WORKDIR, COPY, RUN, and CMD in Dockerfile?

- FROM: Defines the base runtime image (python:3.11-slim).
- WORKDIR: Sets the working directory inside the container (/app).
- COPY: Transfers files from the host context into the container filesystem.
- RUN: Executes commands during build time to install packages and commits immutable layers.
- CMD: Specifies the default command executed when the container starts at runtime.

10. Conclusion

This laboratory assignment successfully demonstrated the end-to-end containerization lifecycle for Big Data workloads. By combining Docker, Docker Compose, Volume bind mounts, and user-defined bridge networks, we built a reproducible, high-throughput, and production-ready data science environment. All source code, visual evidence, and documentation are publicly accessible on GitHub and Docker Hub.